

Chapter 27

Agentic UI Frameworks

As large language models transition from passive text generators to active agents capable of planning, tool use, and multi-step reasoning, the interfaces through which humans interact with them must evolve in parallel. Traditional chat interfaces—designed for single-turn or short-context conversations—are ill-suited to the demands of agentic workflows: long-running tasks, branching decision trees, parallel tool invocations, and the need for meaningful human oversight. This section surveys the landscape of *agentic UI frameworks*: the design paradigms, component libraries, and implementation patterns that enable rich, transparent, and trustworthy human-agent collaboration.

27.1 Motivation: Beyond the Chat Box

Why Agents Need Specialized Interfaces

A chat bubble conveys a *result*. An agentic UI conveys a *process*—the reasoning, the tools invoked, the decisions made, and the points where human judgment is required. Without this visibility, users cannot trust, correct, or learn from the agent.

The gap between a chat interface and an agentic interface mirrors the gap between a vending machine and a skilled collaborator. When an agent executes a 20-step research task, browses the web, writes and runs code, and synthesizes a report, the user needs answers to questions that a simple text response cannot provide:

- **What is the agent doing right now?** Long-running tasks require progress feedback; silence breeds distrust.
- **Why did the agent make this decision?** Transparency into reasoning enables users to catch errors early.
- **Which tools were used, and with what inputs?** Tool provenance is essential for verifying factual claims and auditing behavior.
- **Where should I intervene?** Agents must surface decision points that warrant human judgment without overwhelming users with every micro-decision.
- **Can I undo this?** Irreversible actions (sending emails, modifying files, executing code) require explicit confirmation and rollback paths.

The Automation Bias Risk

Research on human-automation interaction consistently shows that users over-trust automated systems, especially when those systems present outputs confidently and without uncertainty signals [284]. Agentic UIs must actively counteract automation bias by surfacing uncertainty, showing reasoning, and making it easy to question or override agent decisions.

The design of agentic UIs thus sits at the intersection of human-computer interaction (HCI), explainable AI (XAI), and software engineering. The core design goals are:

1. **Transparency:** Make the agent’s internal state legible to the user.
2. **Control:** Provide meaningful intervention points without requiring constant supervision.
3. **Trust Calibration:** Help users develop accurate mental models of agent capabilities and limitations.
4. **Efficiency:** Minimize cognitive load; surface the right information at the right time.
5. **Recoverability:** Make mistakes cheap to detect and reverse.

27.2 UI Paradigms for Agents

No single UI paradigm suits all agentic use cases. The appropriate interface depends on task duration, required human involvement, output type, and user expertise. The spectrum ranges from fully conversational chat interfaces to fully autonomous dashboards with minimal human interaction.

27.2.1 Chat-Based Interfaces

The chat paradigm—message bubbles, a text input, and a scrolling history—remains the most familiar entry point for LLM interaction. Its strengths are low learning curve and natural language flexibility. For agentic use, chat interfaces are augmented with:

- **Streaming responses:** Tokens appear as they are generated, providing immediate feedback and reducing perceived latency. Implemented via Server-Sent Events (SSE) or WebSockets.
- **Inline tool indicators:** Small badges or expandable sections within the message stream show when a tool was called (e.g., “[Searched the web for: climate change 2024]”).
- **Typing indicators and status messages:** “Agent is thinking...”, “Running Python code...”, “Fetching results...” keep users informed during latency gaps.
- **Message threading:** For multi-turn agentic tasks, collapsible sub-threads can contain intermediate steps without cluttering the main conversation.

Chat UI Limitations for Agents

Chat interfaces serialize inherently parallel processes. When an agent fans out to five tools simultaneously, a linear message stream misrepresents the actual execution graph. For complex agentic workflows, chat should be augmented with—or replaced by—richer paradigms.

27.2.2 Canvas and Artifact-Based Interfaces

The canvas paradigm, popularized by Claude Artifacts¹ and ChatGPT Canvas,² introduces a *split-pane* layout: the left pane hosts the conversation, while the right pane (the “canvas” or “artifact panel”) displays generated content—code, documents, diagrams, spreadsheets—as a live, editable artifact.

Key characteristics:

- **Persistent artifacts:** Generated content persists across turns and can be iteratively refined through natural language instructions (“make the chart blue”, “add error handling to the function”).
- **In-place editing:** Users can directly edit the artifact, and the agent can observe and respond to those edits.
- **Version history:** Artifacts maintain a revision history, enabling rollback to any prior state.

- **Multi-artifact workspaces:** Advanced implementations support multiple simultaneous artifacts (e.g., a code file, its test suite, and a documentation page).

The canvas paradigm is particularly well-suited to *co-creation* tasks: writing, coding, data analysis, and design, where the output is a document or artifact rather than a conversational answer.

27.2.3 Workflow Visualization

For agents that execute structured plans—sequences or graphs of steps—workflow visualization UIs make the plan explicit and trackable. This paradigm is common in:

- **Agentic pipelines** (LangGraph, AutoGen, CrewAI): The agent’s execution graph is rendered as a directed acyclic graph (DAG) or flowchart, with nodes representing steps and edges representing data flow or control flow.
- **Task decomposition views:** The agent’s high-level plan is shown as a checklist or Gantt-style timeline, with each sub-task expanding to reveal its own steps.
- **Live progress tracking:** Nodes change color or display spinners as they execute; completed nodes show outputs; failed nodes show error details.

LangGraph Studio3 is the canonical example of this paradigm, providing a graph-based debugger and visualizer for LangGraph agents. Users can inspect the state at each node, replay executions, and inject modified state to test alternative paths.

27.2.4 Dashboard and Monitoring Interfaces

For long-running or production agents, dashboard UIs provide an operational view:

- **Real-time status:** Which agents are running, idle, or failed; current task and step.
- **Resource metrics:** Token consumption, API call counts, latency histograms, cost estimates.
- **Queue management:** Pending tasks, priority ordering, rate limit status.
- **Alert and anomaly detection:** Unusual behavior (excessive retries, cost spikes, repeated failures) surfaced as notifications.
- **Historical analytics:** Task completion rates, average duration, error frequency over time.

Dashboard UIs are typically built with tools like Grafana,⁴ custom React dashboards, or Streamlit, and are aimed at *operators* rather than end users.

27.2.5 Collaborative Interfaces

Collaborative UIs treat the agent as a peer contributor to a shared workspace—a document, codebase, or design canvas—alongside human collaborators. Key features include:

- **Presence indicators:** The agent appears as a named cursor or avatar in the shared workspace.
- **Change attribution:** Edits made by the agent are visually distinguished from human edits (e.g., color-coded diffs).
- **Inline suggestions:** The agent proposes changes as tracked edits or comments, which humans can accept, reject, or modify.
- **Conflict resolution:** When the agent and a human edit the same region simultaneously, the UI surfaces the conflict and facilitates resolution.

This paradigm is emerging in tools like Cursor⁵ (collaborative code editing with AI), Notion AI,⁶ and Google Docs with Gemini integration.⁷

27.2.6 Autonomous with Checkpoints

At the far end of the autonomy spectrum, some agents run largely independently—browsing the web, writing code, executing commands—and surface only at predefined *checkpoints* requiring human approval. This paradigm is used in:

- **Computer-use agents** (Anthropic Computer Use,⁸ OpenAI Operator⁹): The agent controls a browser or desktop; the UI shows a live screen feed and pauses for approval before irreversible actions.
- **Automated pipelines with gates**: CI/CD-style workflows where the agent completes a phase and waits for a human “merge” before proceeding.
- **Scheduled agents**: Agents that run on a schedule and report results asynchronously, with a notification-based UI for reviewing outputs and approving follow-on actions.

Checkpoint UI in Practice

An agent tasked with “clean up my email inbox” might autonomously categorize and archive 500 emails, then pause and present a summary: “I found 23 emails that appear to be from mailing lists you haven’t opened in 6 months. Shall I unsubscribe from all, some, or none?” The user reviews a list, makes selections, and the agent proceeds. This pattern—autonomous execution punctuated by human decision points—balances efficiency with control.

27.3 Key UI Components for Agents

Regardless of the overarching paradigm, agentic UIs share a set of recurring components. This section catalogs the most important, with design guidance for each.

27.3.1 Thought and Reasoning Display

Modern LLMs, particularly those trained with chain-of-thought or extended thinking (e.g., OpenAI o1/o3, Anthropic Claude with extended thinking), generate substantial internal reasoning before producing a final response. Surfacing this reasoning is a double-edged sword: it increases transparency but can overwhelm users with verbose internal monologue.

Best practices:

- **Collapsible reasoning blocks**: Show a summary (“Thought for 12 seconds”) with an expand toggle for users who want details.
- **Progressive disclosure**: Show only the final conclusion by default; reasoning is available on demand.
- **Structured reasoning**: If the model produces structured thoughts (hypotheses, evidence, conclusions), render them with visual hierarchy rather than as a wall of text.
- **Reasoning vs. response distinction**: Clearly visually distinguish internal reasoning (which may contain errors or false starts) from the final response.

27.3.2 Tool Use Visualization

Tool calls are the primary mechanism by which agents interact with the world. Visualizing them is essential for trust and debugging.

Tool Call Anatomy

Each tool invocation has four components worth displaying: (1) the **tool name** and icon, (2) the **input arguments** (potentially large JSON), (3) the **output/result** (potentially large), and (4) **timing** (latency). The UI must balance completeness with readability.

Design patterns for tool visualization:

- **Inline tool cards:** Compact cards within the message stream showing tool name, a one-line summary of inputs, and status (running/success/error). Expandable for full details.
- **Tool timeline:** A horizontal timeline showing all tool calls in a turn, with durations, enabling identification of bottlenecks.
- **Input/output diff:** For tools that modify state (e.g., file editing), show a before/after diff.
- **Tool icons and branding:** Recognizable icons for common tools (web search, code execution, file system, APIs) enable rapid scanning.
- **Error highlighting:** Failed tool calls shown in red with the error message and any retry attempts.

27.3.3 Progress Indicators

Multi-step agentic tasks require rich progress feedback:

- **Step-level progress:** A numbered list of planned steps with checkmarks as each completes. For dynamic plans, steps can be added or removed as the agent adapts.
- **Token streaming indicators:** A blinking cursor or animated ellipsis during generation; a token-per-second counter for power users.
- **Estimated completion:** Where feasible, an ETA based on task complexity and historical performance. Displayed with appropriate uncertainty (“approximately 2–5 minutes”).
- **Subtask nesting:** For hierarchical tasks, a tree-structured progress view with expandable subtasks.
- **Cancellation:** A clearly visible “Stop” button that gracefully halts the agent and summarizes work completed so far.

27.3.4 Approval Gates

Approval gates are the primary mechanism for human-in-the-loop control. They must be designed to be *informative* (giving users enough context to make a good decision) without being *fatiguing* (requiring approval for every trivial action).

Alert Fatigue in Approval Gates

If an agent requests approval too frequently, users will begin approving reflexively without reading—defeating the purpose of the gate. Tiered approval policies (see Section 27.7) are essential to maintain meaningful oversight.

Approval gate UI elements:

- **Action summary:** Plain-language description of what the agent wants to do (“Send an email to john@example.com with the attached report”).
- **Risk indicator:** Visual signal of action reversibility (green = easily undoable, yellow = hard to undo, red = irreversible).
- **Approve / Reject / Modify:** Three-option interface; “Modify” opens an editor for the action parameters before approval.
- **Context panel:** Expandable section showing why the agent wants to take this action (relevant reasoning, prior steps).
- **Timeout behavior:** Clear indication of what happens if the user doesn’t respond (agent pauses, not proceeds).

27.3.5 Context Display

Agents maintain internal state—memory, active tools, retrieved documents, conversation history—that influences their behavior. Making this state visible helps users understand and predict agent behavior.

- **Memory panel:** Shows what the agent currently “remembers” about the user, task, and prior interactions. Editable by the user.
- **Active tools list:** Which tools are currently available to the agent, with enable/disable toggles.
- **Retrieved context:** Documents or data chunks currently in the agent’s context window, with source citations.
- **Token budget indicator:** How much of the context window is consumed, helping users understand when to start a new session.

27.3.6 Error and Recovery UI

Agents fail—tools return errors, models hallucinate, plans become infeasible. The UI must handle failures gracefully:

- **Error cards:** Inline display of failures with the error type, message, and the agent’s interpretation.
- **Retry controls:** Manual retry button with optional parameter adjustment.
- **Alternative approaches:** When the primary approach fails, the agent proposes alternatives; the UI presents them as selectable options.
- **Partial results:** If a multi-step task fails midway, the UI shows completed steps and their outputs, preserving partial value.
- **Escalation path:** A clear path to human support or manual completion when the agent cannot proceed.

27.3.7 Confidence Indicators

LLMs are probabilistic systems with calibrated (or miscalibrated) uncertainty. Surfacing confidence helps users know when to trust and when to verify:

- **Verbal hedging display:** Highlight phrases like “I’m not certain” or “you may want to verify” to draw attention to low-confidence claims.
- **Source quality indicators:** For retrieved information, show source recency, authority, and relevance scores.
- **Explicit uncertainty requests:** A “How confident are you?” button that prompts the agent to self-assess and explain its uncertainty.
- **Verification suggestions:** For high-stakes outputs, the agent proactively suggests verification steps (“I recommend checking this calculation independently”).

27.4 Frameworks and Libraries

A growing ecosystem of frameworks accelerates the development of agentic UIs. We survey the most widely adopted, organized by primary language and use case.

27.4.1 Vercel AI SDK

The Vercel AI SDK [358] is a TypeScript/JavaScript library for building streaming AI interfaces in React, Next.js, Svelte, and Vue. It is the most widely used framework for production web-based agent UIs.

Core abstractions:

- **useChat:** A React hook managing a chat conversation with streaming support, message history, and loading states.
- **useCompletion:** A hook for single-turn text completion with streaming.
- **useObject:** Streams structured JSON objects, enabling progressive rendering of complex outputs.
- **streamText / streamObject:** Server-side functions that stream LLM responses over HTTP.

Generative UI (AI SDK RSC): The most distinctive feature of the Vercel AI SDK is its support for *generative UI* via React Server Components (RSC). Rather than returning text, the LLM can invoke tools whose results are rendered as arbitrary React components—a weather widget, a stock chart, a booking form—streamed directly into the UI. This is discussed further in Section 27.5.

27.4.2 Chainlit

Chainlit [40] is a Python framework for building production-ready agent UIs with minimal boilerplate. It is particularly popular in the LangChain and LlamaIndex ecosystems.

Key features:

- **Step visualization:** Chainlit natively renders LangChain and LlamaIndex execution steps as a collapsible tree, showing each chain call, retrieval, and tool invocation.
- **Multi-modal support:** File uploads, image display, audio playback, and PDF rendering out of the box.
- **Authentication and sessions:** Built-in user authentication, persistent conversation history, and multi-user support.
- **Custom elements:** React components can be registered and rendered from Python, enabling rich custom visualizations.
- **Feedback collection:** Built-in thumbs up/down feedback with optional comments, stored to a database.

```
import chainlit as cl
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

agent = create_react_agent(
    ChatOpenAI(model="gpt-4o"), tools=[search]
)

@cl.on_message
async def on_message(message: cl.Message):
    # Chainlit automatically renders each step as a collapsible UI element
```

```
# when using the callback handler
async with cl.Step(name="Agent", type="run") as step:
    step.input = message.content
    result = await agent.ainvoke(
        {"messages": [{ "role": "user", "content": message.content }]},
        config={"callbacks": [cl.LangchainCallbackHandler()]}
    )
    output = result["messages"][-1].content
    step.output = output

    await cl.Message(content=output).send()
```

Listing 27.1: Minimal Chainlit agent with step visualization

27.4.3 Gradio

Gradio [1] is a Python library for rapidly building ML demos and agent interfaces. Its `gr.ChatInterface` and `gr.Blocks` API enable quick prototyping of conversational agents with minimal code.

Strengths for agentic UIs:

- **Zero-configuration deployment:** One-line sharing via Hugging Face Spaces.
- **Custom components:** The Gradio Custom Components system allows building React components that integrate seamlessly with Python backends.
- **Multi-modal inputs:** File upload, image, audio, video, and webcam inputs with minimal configuration.
- **Streaming:** Native support for generator-based streaming responses.

Limitations: Gradio's layout system is less flexible than full React frameworks, and its state management is session-scoped, making complex multi-agent coordination challenging.

27.4.4 Streamlit

Streamlit [156] is a Python framework for data applications that has been widely adopted for agent dashboards and monitoring UIs. Its reactive execution model—the entire script reruns on each interaction—is simple but can be limiting for complex agentic workflows.

Agentic use cases:

- **Agent dashboards:** Real-time metrics, task queues, and status displays using `st.metric`, `st.dataframe`, and `st.status`.
- **Session state:** `st.session_state` persists agent state across reruns, enabling multi-turn conversations.
- **Streaming:** `st.write_stream` renders generator outputs progressively.
- **Fragments:** `@st.fragment` decorator enables partial reruns, improving performance for live-updating dashboards.

27.4.5 OpenAI Assistants Playground

The OpenAI Assistants Playground serves as a reference implementation for agentic UI design. It demonstrates:

- Thread-based conversation management with persistent history.
- File attachment and retrieval visualization.

- Code interpreter execution with output display (stdout, images, files).
- Function call display with input/output inspection.
- Run step visualization showing the sequence of model calls and tool invocations.

While not a framework for building custom UIs, the Playground’s design patterns are widely emulated.

27.4.6 LangGraph Studio

LangGraph Studio [155] is a desktop application providing a visual IDE for LangGraph agents. It is the most sophisticated tool-use and workflow visualization environment currently available.

Features:

- **Graph visualization:** Interactive rendering of the agent’s state machine, with nodes representing agent steps and edges representing transitions.
- **State inspection:** At any point in execution, the full agent state (all variables, memory, tool results) can be inspected as structured JSON.
- **Time-travel debugging:** Replay any prior execution step, modify the state, and re-run from that point.
- **Human-in-the-loop integration:** Breakpoints can be set on any node; execution pauses and waits for human input before proceeding.
- **Multi-agent support:** Visualizes supervisor-subagent hierarchies and inter-agent message passing.

27.4.7 Framework Comparison

Table 27.1 summarizes the key characteristics of the frameworks discussed above.

Table 27.1: Agentic UI framework comparison.

Framework	Language	Stream	Tool Viz	Multi-Ag.	Gen UI	Prod
Vercel AI SDK	TypeScript	✓	Partial	Partial	✓	✓
Chainlit	Python	✓	✓	Partial	Partial	✓
Gradio	Python	✓	○	×	○	✓
Streamlit	Python	✓	○	×	×	✓
OAI Playground	N/A (hosted)	✓	✓	×	×	×
LangGraph Studio	Python/TS	✓	✓	✓	×	Partial

27.5 Generative UI

The Generative UI Concept

Traditional LLM interfaces render model outputs as text or markdown. *Generative UI* inverts this: the model’s tool calls *generate* UI components. The model decides not just *what* to say, but *how* to present it—as a chart, a form, a map, a calendar widget—based on the content type and user intent.

Generative UI represents a fundamental shift in the relationship between LLMs and interfaces. Rather than the developer pre-specifying all possible UI states, the model dynamically selects and parameterizes UI components appropriate to the current context.

27.5.1 React Server Components for Dynamic Interfaces

The Vercel AI SDK's RSC (React Server Components¹⁰) integration is the most mature implementation of generative UI. The architecture works as follows:

1. The user sends a message to a Next.js¹¹ server action.
2. The server calls the LLM with a set of tools, each associated with a React component.
3. When the LLM calls a tool (e.g., `show_weather`), the server renders the corresponding React component with the tool's output as props.
4. The rendered component is streamed to the client as a React Server Component, appearing inline in the chat.

```
// app/actions.tsx (Server Action)
import { streamUI } from 'ai/rsc';
import { openai } from '@ai-sdk/openai';
import { WeatherCard } from '@components/WeatherCard';
import { StockChart } from '@components/StockChart';

export async function chat(userMessage: string) {
  const result = await streamUI({
    model: openai('gpt-4o'),
    messages: [{ role: 'user', content: userMessage }],
    tools: {
      show_weather: {
        description: 'Display current weather for a location',
        parameters: z.object({
          location: z.string(),
          unit: z.enum(['celsius', 'fahrenheit']),
        }),
        // Tool result rendered as a React component
        generate: async ({ location, unit }) => {
          const data = await fetchWeather(location, unit);
          return <WeatherCard data={data} />;
        },
      },
      show_stock: {
        description: 'Display stock price chart',
        parameters: z.object({ ticker: z.string() }),
        generate: async ({ ticker }) => {
          const data = await fetchStockData(ticker);
          return <StockChart ticker={ticker} data={data} />;
        },
      },
    },
  });
  return result.value;
}
```

Listing 27.2: Generative UI with Vercel AI SDK RSC (TypeScript)

27.5.2 Adaptive Interfaces Based on Content Type

Generative UI enables interfaces that adapt to the nature of the content being presented:

- **Tabular data** → sortable, filterable data table with export options.
- **Geographic data** → interactive map with markers and layers.
- **Time series** → zoomable line chart with annotations.

- **Code** → syntax-highlighted editor with run button.
- **Documents** → formatted document viewer with annotation tools.
- **Forms/structured input** → dynamically generated form fields.

The model acts as a *UI orchestrator*, selecting the most appropriate presentation for each piece of information. This reduces the need for developers to anticipate every possible output type and pre-build corresponding components.

Limits of Generative UI

How much UI generation should be delegated to the model? Fully model-driven UI risks inconsistency, accessibility failures, and security vulnerabilities (e.g., a model generating a form that submits data to an unexpected endpoint). In practice, generative UI works best when the model selects from a *curated library* of pre-built, accessible, and secure components rather than generating arbitrary HTML or JSX.

27.6 Streaming and Real-Time Patterns

Streaming is foundational to agentic UIs: it transforms the experience from “wait for a result” to “watch the agent work.” This section covers the key streaming patterns and their implementation considerations.

27.6.1 Token Streaming

Token streaming delivers LLM output incrementally as tokens are generated, rather than waiting for the complete response. Two transport mechanisms are commonly used:

- **Server-Sent Events (SSE)**¹²: A unidirectional HTTP stream from server to client. Each event carries a chunk of tokens. SSE is simple, works over standard HTTP/1.1, and is automatically reconnected by browsers. It is the dominant mechanism for LLM streaming APIs (OpenAI, Anthropic, Google all use SSE).
- **WebSockets**: Bidirectional persistent connections. More complex to implement but necessary for interactive streaming scenarios where the client needs to send data mid-stream (e.g., interrupting the agent, providing mid-generation feedback).

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from openai import AsyncOpenAI
import json

app = FastAPI()
client = AsyncOpenAI()

async def token_stream(prompt: str):
    """Generator that yields SSE-formatted token chunks."""
    stream = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )
    async for chunk in stream:
        delta = chunk.choices[0].delta
        if delta.content:
            # SSE format: "data: <json>\n\n"
            yield f"data: {json.dumps({'token': delta.content})}\n\n"
        elif chunk.choices[0].finish_reason:
            yield f"data: {json.dumps({'done': True})}\n\n"
```

```
@app.get("/stream")
async def stream_endpoint(prompt: str):
    return StreamingResponse(
        token_stream(prompt),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
```

Listing 27.3: SSE token streaming with FastAPI

27.6.2 Tool Call Streaming

Modern LLM APIs support streaming tool calls: the tool name and arguments are streamed incrementally, enabling the UI to show “Agent is calling `search_web` with query: ‘climate change 2024’...” before the tool has even been invoked. This requires parsing partial JSON, which can be done with streaming JSON parsers.

Patterns for tool call streaming:

- **Progressive argument display:** Show tool arguments as they stream in, even before the call is complete.
- **Parallel tool call indicators:** When the model calls multiple tools simultaneously, show all of them as pending, then update each as results arrive.
- **Tool result streaming:** Some tools (e.g., code execution, web scraping) can themselves stream results; pipe these through to the UI progressively.

27.6.3 Multi-Agent Streaming

In multi-agent systems, multiple agents may be generating output simultaneously. The UI must handle parallel streams:

- **Agent-labeled streams:** Each stream is tagged with the agent’s identity; the UI renders them in separate lanes or panels.
- **Stream merging:** For supervisor-subagent patterns, the supervisor’s stream may interleave with subagent streams; the UI must maintain coherent ordering.
- **Backpressure:** If the UI cannot render as fast as streams arrive (e.g., multiple agents generating simultaneously), a backpressure mechanism must prevent buffer overflow. Strategies include: dropping intermediate tokens (showing only the latest), batching updates, or pausing slower streams.

27.6.4 Optimistic UI Updates

Optimistic UI updates improve perceived responsiveness by immediately reflecting user actions in the UI before server confirmation:

- When a user sends a message, it appears immediately in the chat history (optimistically) while the request is in flight.
- When an approval gate is accepted, the UI immediately shows the action as “approved” and begins showing the agent’s next steps, even before the server has processed the approval.
- If the server returns an error, the optimistic update is rolled back and an error state is shown.

27.6.5 Backpressure Handling

In high-throughput agentic scenarios, the rate of incoming data can exceed the UI’s rendering capacity. Strategies for managing backpressure:

- **Token batching:** Buffer tokens for 50–100ms and render in batches rather than one-by-one, reducing DOM update frequency.
- **Virtual scrolling:** For long outputs, render only the visible portion of the content, discarding off-screen DOM nodes.
- **Throttled updates:** For metrics and status displays, update at a fixed rate (e.g., 10 Hz) regardless of the incoming data rate.
- **Progressive detail:** Show a summary view during high-throughput periods; full detail available on demand.

27.7 Human-in-the-Loop UI Design

Human-in-the-loop (HITL) interaction is one of the most consequential design challenges in agentic UIs. The goal is to maintain meaningful human oversight without creating a bottleneck that negates the efficiency benefits of automation.

27.7.1 When to Interrupt the Agent

Not all agent actions warrant human review. A principled interruption policy considers:

- **Reversibility:** Irreversible actions (deleting files, sending emails, making purchases) always warrant approval. Reversible actions (reading files, searching the web) generally do not.
- **Scope:** Actions affecting external systems or other people warrant more scrutiny than purely local actions.
- **Confidence:** When the agent’s confidence in its interpretation of the user’s intent is low, it should ask for clarification rather than proceed.
- **Cost:** High-cost actions (large API calls, expensive computations) warrant approval.
- **Novelty:** Actions the agent has not taken before in this context warrant more scrutiny than routine actions.

27.7.2 Tiered Approval Workflows

A tiered approval policy balances oversight with efficiency:

Three-Tier Approval Model

Tier 1 (Auto-approve): Low-risk, reversible, routine actions. Examples: web search, reading files, calling read-only APIs. The agent proceeds without interruption; actions are logged for audit.

Tier 2 (Notify): Medium-risk actions. The UI shows a non-blocking notification (“Agent sent a draft email to your Drafts folder”) that the user can review asynchronously. A brief window (e.g., 30 seconds) allows cancellation before the action is finalized.

Tier 3 (Require approval): High-risk, irreversible, or high-cost actions. The agent pauses and presents a blocking approval gate. The user must explicitly approve, reject, or modify before the agent continues.

The thresholds between tiers can be configured by the user (“always ask before sending emails”) or learned from user behavior (if the user always approves web searches, auto-approve them in the future).

27.7.3 Feedback Mechanisms

Beyond approval gates, agentic UIs should provide rich feedback mechanisms that help the agent improve over time:

- **Thumbs up/down:** Simple binary feedback on responses, stored and used for RLHF fine-tuning or preference learning.
- **Inline corrections:** Users can directly edit agent outputs; the delta between the original and corrected output is a training signal.
- **Preference selection:** When the agent offers multiple options, the user’s selection is a preference signal.
- **Explicit instruction:** “Don’t do this again”, “Always ask before X”, “Prefer approach Y over Z”—natural language instructions that update the agent’s behavioral policy.
- **Rating with rationale:** Optional free-text explanation accompanying a rating, providing richer signal than binary feedback.

27.7.4 Teaching the Agent Through UI Interaction

The most sophisticated HITL UIs treat every interaction as a teaching opportunity:

- **Demonstration:** The user performs a task manually; the agent observes and learns the preferred approach.
- **Correction with generalization:** When the user corrects an agent action, the UI asks “Should I always do this differently?” to generalize the correction.
- **Preference elicitation:** Periodic prompts asking the user to compare two agent behaviors and indicate which is preferred.
- **Behavioral profiles:** The UI maintains a visible “preferences” profile that the user can review and edit, making the agent’s learned behaviors transparent and controllable.

27.8 Accessibility and Trust

Trust is not a feature—it is an emergent property of a system that consistently behaves as expected, explains itself clearly, and recovers gracefully from failures. Agentic UIs must be designed with trust as a first-class concern.

27.8.1 Explaining Agent Decisions

Explainability in agentic UIs goes beyond showing chain-of-thought. It requires:

- **Decision rationale:** For consequential decisions, the agent should explain not just *what* it decided but *why*—which factors were considered, what alternatives were rejected, and what assumptions were made.
- **Source attribution:** Claims should be linked to their sources; retrieved documents should be citable.
- **Counterfactual explanations:** “If you had said X instead of Y, I would have done Z”—helping users understand the agent’s decision boundary.
- **Uncertainty quantification:** Explicit statements of confidence, with the factors driving uncertainty.

27.8.2 Showing Confidence Levels

Confidence indicators must be calibrated and meaningful:

- **Verbal confidence:** Natural language expressions (“I’m fairly confident”, “I’m not sure about this”) are more interpretable than numerical probabilities for most users.
- **Visual confidence:** Color coding (green/yellow/red), icon variants, or font weight can encode confidence without adding text.
- **Confidence by claim:** For multi-claim responses, per-claim confidence indicators (e.g., inline footnotes) are more informative than a single response-level score.

27.8.3 Undo and Rollback Capabilities

Every consequential agent action should be undoable where technically feasible:

- **Action log with undo:** A chronological log of all agent actions with an “Undo” button for each reversible action.
- **Snapshot-based rollback:** For stateful tasks (e.g., code editing, document writing), periodic snapshots enable rollback to any prior state.
- **Dry-run mode:** Before executing a plan, the agent can simulate it and show the predicted state changes, allowing the user to approve or modify before any real action is taken.
- **Graceful degradation:** When an undo is not possible (e.g., an email has been sent), the UI clearly communicates this and offers the best available alternative (e.g., sending a follow-up).

27.8.4 Audit Trails in the UI

For enterprise and regulated use cases, audit trails are essential:

- **Immutable action log:** Every agent action, tool call, and human approval is logged with timestamp, user identity, and full parameters.
- **Exportable history:** The audit trail can be exported as JSON, CSV, or PDF for compliance reporting.
- **Diff views:** For document or code modifications, the audit trail includes before/after diffs.
- **Session replay:** The ability to replay an entire agent session, step by step, for debugging or compliance review.

27.8.5 Managing User Expectations

Miscalibrated expectations are a primary source of user distrust. Agentic UIs should actively manage expectations:

- **Capability disclosure:** Clear, accessible documentation of what the agent can and cannot do.
- **Limitation acknowledgment:** When the agent encounters a task outside its capabilities, it says so clearly rather than attempting and failing silently.
- **Uncertainty communication:** Proactive communication of uncertainty, rather than waiting for the user to discover errors.
- **Consistent persona:** A consistent agent identity and communication style builds familiarity and predictability.

Trust-Building Through Transparency: A Case Study

Consider an agent tasked with booking a flight. A low-trust UI presents: “I’ve booked your flight. Confirmation: AA1234.” A high-trust UI presents: (1) a summary of the search parameters used, (2) the alternatives considered and why this flight was selected, (3) the exact actions taken (API calls to the booking system), (4) the confirmation details with a link to the booking, (5) an undo option valid for the next 24 hours, and (6) a note about what the agent cannot do (e.g., “I cannot modify this booking; you’ll need to call the airline directly”). The second UI takes more screen space but builds the user’s confidence that the agent acted correctly and gives them the information needed to verify and recover if needed.

27.9 Implementation Example: A Full-Stack Agentic UI

We now present a concrete implementation example combining streaming, tool visualization, and approval gates in a Python/React stack. The backend uses FastAPI with LangGraph; the frontend uses React with the Vercel AI SDK patterns adapted for a custom backend.

27.9.1 Backend: FastAPI + LangGraph with Streaming and Approval Gates

```
# backend/main.py
import asyncio
import json
from typing import AsyncGenerator
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

app = FastAPI()

# -- Tool definitions -----

@tool
def web_search(query: str) -> str:
    """Search the web for information."""
    return f"Search results for '{query}': [simulated results]"

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email. REQUIRES HUMAN APPROVAL."""
    return f"Email sent to {to} with subject '{subject}'"

@tool
def read_file(path: str) -> str:
    """Read a file from the filesystem."""
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: File not found: {path}"

# Tools requiring approval (Tier 3)
APPROVAL_REQUIRED_TOOLS = {"send_email"}

# -- Approval gate store (in-memory; use Redis in production) -----

approval_store: dict[str, asyncio.Event] = {}
approval_results: dict[str, dict] = {}

# -- LLM setup -----
```



```

llm = ChatOpenAI(model="gpt-4o", streaming=True)
tools = [web_search, send_email, read_file]
llm_with_tools = llm.bind_tools(tools)

def should_request_approval(tool_name: str) -> bool:
    return tool_name in APPROVAL_REQUIRED_TOOLS

# -- Streaming endpoint -----

async def agent_stream(
    session_id: str,
    user_message: str,
) -> AsyncGenerator[str, None]:
    """Stream agent events as SSE."""

    def sse(event_type: str, data: dict) -> str:
        return f"data: {json.dumps({'type': event_type, **data})}\n\n"

    yield sse("status", {"message": "Agent starting..."})

    # Simulate multi-step agent execution
    steps = [
        ("thinking", {"content": "Analyzing the request..."}),
        ("tool_call", {
            "tool": "web_search",
            "input": {"query": user_message},
            "tier": 1, # Auto-approve
        }),
        ("tool_result", {
            "tool": "web_search",
            "output": f"Results for: {user_message}",
            "duration_ms": 342,
        }),
    ]

    for event_type, data in steps:
        await asyncio.sleep(0.5) # Simulate processing time
        yield sse(event_type, data)

    # Simulate a Tier 3 action requiring approval
    approval_id = f"{session_id}_email_001"
    approval_event = asyncio.Event()
    approval_store[approval_id] = approval_event

    yield sse("approval_required", {
        "approval_id": approval_id,
        "tool": "send_email",
        "tier": 3,
        "risk": "irreversible",
        "action_summary": "Send summary email to user@example.com",
        "parameters": {
            "to": "user@example.com",
            "subject": f"Research results: {user_message}",
            "body": "Here are the findings...",
        },
    })

    # Wait for human approval (timeout after 5 minutes)
    try:
        await asyncio.wait_for(approval_event.wait(), timeout=300)
        result = approval_results.get(approval_id, {})

        if result.get("approved"):
            yield sse("tool_call", {
                "tool": "send_email",
                "input": result.get("parameters", {}),

```

```

        "tier": 3,
        "approved_by": "human",
    })
    await asyncio.sleep(0.3)
    yield sse("tool_result", {
        "tool": "send_email",
        "output": "Email sent successfully",
        "duration_ms": 128,
    })
else:
    yield sse("action_rejected", {
        "tool": "send_email",
        "reason": result.get("reason", "User rejected"),
    })
except asyncio.TimeoutError:
    yield sse("approval_timeout", {
        "approval_id": approval_id,
        "message": "Approval timed out; action skipped",
    })
})

# Final response
yield sse("token", {"content": "I've completed the research. "})
yield sse("token", {"content": "Here's a summary of what I found..."})
yield sse("done", {"total_tokens": 847, "duration_ms": 2341})

@app.get("/chat/stream")
async def chat_stream(session_id: str, message: str):
    return StreamingResponse(
        agent_stream(session_id, message),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )

class ApprovalRequest(BaseModel):
    approval_id: str
    approved: bool
    parameters: dict | None = None
    reason: str | None = None

@app.post("/chat/approve")
async def handle_approval(req: ApprovalRequest):
    if req.approval_id not in approval_store:
        raise HTTPException(status_code=404, detail="Approval not found")
    approval_results[req.approval_id] = {
        "approved": req.approved,
        "parameters": req.parameters,
        "reason": req.reason,
    }
    approval_store[req.approval_id].set()
    return {"status": "ok"}

```

Listing 27.4: FastAPI backend with streaming and approval gates

27.9.2 Frontend: React with Streaming and Tool Visualization

```

// frontend/AgentChat.tsx
import { useState, useEffect, useRef } from 'react';

// -- Types -----
type AgentEvent =
  | { type: 'status'; message: string }
  | { type: 'thinking'; content: string }
  | { type: 'token'; content: string }
  | { type: 'tool_call'; tool: string; input: object; tier: number }

```

```

| { type: 'tool_result'; tool: string; output: string; duration_ms: number }
| { type: 'approval_required'; approval_id: string; tool: string;
  tier: number; risk: string; action_summary: string; parameters: object }
| { type: 'action_rejected'; tool: string; reason: string }
| { type: 'done'; total_tokens: number; duration_ms: number };

// -- Tool Card Component -----

function ToolCard({ event }: { event: AgentEvent & { type: 'tool_call' } }) {
  const [expanded, setExpanded] = useState(false);
  const tierColors = { 1: '#22c55e', 2: '#f59e0b', 3: '#ef4444' };
  const color = tierColors[event.tier as keyof typeof tierColors] || '#6b7280';

  return (
    <div style={{ border: '1px solid ${color}', borderRadius: 8, padding: 8,
      margin: '4px 0', fontSize: 13 }}>
      <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
        <span style={{ color, fontWeight: 600 }}>[gear] {event.tool}</span>
        <span style={{ color: '#6b7280', fontSize: 11 }}>
          Tier {event.tier} | {event.tier === 1 ? 'Auto' : 'Approved'}
        </span>
        <button onClick={() => setExpanded(!expanded)}
          style={{ marginLeft: 'auto', fontSize: 11 }}>
          {expanded ? 'Hide' : 'Details'}
        </button>
      </div>
      {expanded && (
        <pre style={{ marginTop: 8, fontSize: 11, background: '#f3f4f6',
          padding: 8, borderRadius: 4, overflow: 'auto' }}>
          {JSON.stringify(event.input, null, 2)}
        </pre>
      )}
    </div>
  );
}

// -- Approval Gate Component -----

function ApprovalGate({
  event,
  onDecision,
}: {
  event: AgentEvent & { type: 'approval_required' };
  onDecision: (approved: boolean, params?: object) => void;
}) {
  const riskColors = { reversible: '#22c55e', 'hard-to-undo': '#f59e0b',
    irreversible: '#ef4444' };
  const riskColor = riskColors[event.risk as keyof typeof riskColors] || '#6b7280';

  return (
    <div style={{ border: '2px solid ${riskColor}', borderRadius: 8,
      padding: 16, margin: '8px 0', background: '#fef9f0' }}>
      <div style={{ fontWeight: 700, color: riskColor, marginBottom: 8 }}>
        [!] Approval Required: {event.tool}
      </div>
      <div style={{ marginBottom: 8 }}>{event.action_summary}</div>
      <div style={{ fontSize: 12, color: '#6b7280', marginBottom: 12 }}>
        Risk level: <span style={{ color: riskColor }}>{event.risk}</span>
      </div>
      <div style={{ display: 'flex', gap: 8 }}>
        <button
          onClick={() => onDecision(true, event.parameters)}
          style={{ background: '#22c55e', color: 'white', border: 'none',
            borderRadius: 6, padding: '8px 16px', cursor: 'pointer' }}>
          [ok] Approve

```

```

    </button>
    <button
      onClick={() => onDecision(false)}
      style={{ background: '#ef4444', color: 'white', border: 'none',
        borderRadius: 6, padding: '8px 16px', cursor: 'pointer' }}>
      [x] Reject
    </button>
  </div>
</div>
);
}

// -- Main Chat Component -----

export function AgentChat() {
  const [events, setEvents] = useState<AgentEvent[]>([]);
  const [response, setResponse] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [input, setInput] = useState('');
  const sessionId = useRef(`session_${Date.now()}`);

  const sendMessage = async () => {
    if (!input.trim() || isStreaming) return;
    setEvents([]);
    setResponse('');
    setIsStreaming(true);

    const url = `/chat/stream?session_id=${sessionId.current}`
      + `&message=${encodeURIComponent(input)}`;
    const es = new EventSource(url);

    es.onmessage = (e) => {
      const event: AgentEvent = JSON.parse(e.data);
      if (event.type === 'token') {
        setResponse(prev => prev + event.content);
      } else if (event.type === 'done') {
        setIsStreaming(false);
        es.close();
      } else {
        setEvents(prev => [...prev, event]);
      }
    };

    es.onerror = () => { setIsStreaming(false); es.close(); };
    setInput('');
  };

  const handleApproval = async (
    approvalId: string,
    approved: boolean,
    parameters?: object,
  ) => {
    await fetch('/chat/approve', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ approval_id: approvalId, approved, parameters }),
    });
  };

  return (
    <div style={{ maxWidth: 800, margin: '0 auto', padding: 16 }}>
      <div style={{ minHeight: 400, border: '1px solid #e5e7eb',
        borderRadius: 8, padding: 16, marginBottom: 16 }}>
        {events.map((event, i) => {
          if (event.type === 'tool_call')
            return <ToolCard key={i} event={event} />;
        })}
      </div>
    </div>
  );
}

```

```

    if (event.type === 'approval_required')
      return (
        <ApprovalGate key={i} event={event}
          onDecision={({approved, params}) =>
            handleApproval(event.approval_id, approved, params)} />
        );
    if (event.type === 'status' || event.type === 'thinking')
      return (
        <div key={i} style={{ color: '#6b7280', fontSize: 12,
          fontStyle: 'italic', margin: '4px 0' }}>
          {event.type === 'thinking' ? event.content : event.message}
        </div>
      );
    return null;
  }}}
  {response && (
    <div style={{ marginTop: 8, lineHeight: 1.6 }}>
      {response}
      {isStreaming && <span className="cursor-blink">|</span>}
    </div>
  )}
</div>
<div style={{ display: 'flex', gap: 8 }}>
  <input
    value={input}
    onChange={e => setInput(e.target.value)}
    onKeyDown={e => e.key === 'Enter' && sendMessage()}
    placeholder="Ask the agent..."
    style={{ flex: 1, padding: '8px 12px', borderRadius: 6,
      border: '1px solid #d1d5db', fontSize: 14 }}
  />
  <button onClick={sendMessage} disabled={isStreaming}
    style={{ padding: '8px 16px', background: '#3b82f6',
      color: 'white', border: 'none', borderRadius: 6,
      cursor: isStreaming ? 'not-allowed' : 'pointer' }}>
    {isStreaming ? 'Running...' : 'Send'}
  </button>
</div>
</div>
);
}

```

Listing 27.5: React frontend with streaming tool visualization and approval gates

What This Implementation Demonstrates

The code above illustrates several key agentic UI patterns working together:

- **SSE streaming:** The backend streams events of different types (status, thinking, tool calls, tokens) over a single HTTP connection.
- **Typed event protocol:** A discriminated union of event types enables the frontend to render each event appropriately.
- **Tool visualization:** ToolCard renders tool calls with tier indicators and expandable input details.
- **Approval gates:** ApprovalGate blocks the stream and waits for human input before the agent proceeds with irreversible actions.
- **Async approval:** The backend uses `asyncio.Event` to pause the stream while waiting for the frontend's approval POST request, cleanly decoupling the approval UI from the streaming logic.

27.10 Summary

Agentic UI frameworks represent a new frontier in human-computer interaction, demanding a rethinking of interface design from first principles. The key insights from this section are:

1. **Paradigm selection matters:** The appropriate UI paradigm (chat, canvas, workflow, dashboard, collaborative, autonomous) depends on task structure, required human involvement, and output type. Most production systems combine multiple paradigms.
2. **Transparency is non-negotiable:** Users cannot trust what they cannot see. Thought display, tool visualization, and context panels are not optional features—they are the foundation of trustworthy agentic systems.
3. **Streaming is the baseline:** Users expect to see agents work in real time. Token streaming, tool call streaming, and multi-agent streaming are table-stakes capabilities.
4. **Approval gates must be tiered:** Flat approval policies (approve everything or approve nothing) fail in practice. Tiered policies that auto-approve safe actions and gate dangerous ones maintain oversight without creating bottlenecks.
5. **Generative UI is the frontier:** The ability for LLMs to generate not just text but UI components—charts, forms, maps, widgets—enables interfaces that adapt to content rather than forcing content into a fixed template.
6. **Trust is earned through consistency and recoverability:** Undo capabilities, audit trails, and calibrated confidence indicators are as important as raw capability for building user trust.

Design Principle: The Agent as a Transparent Collaborator

The north star for agentic UI design is the *transparent collaborator*: an agent whose actions are always visible, whose reasoning is always accessible, whose mistakes are always recoverable, and whose capabilities and limitations are always clear. Every UI decision should be evaluated against this standard.

The frameworks and patterns described in this section—Vercel AI SDK, Chainlit, Gradio, Streamlit, LangGraph Studio—provide the building blocks. The challenge for practitioners is to combine them thoughtfully, guided by the specific needs of their users and the specific risks of their domain.